

Docker Architecture and Technologies

<https://docs.docker.com/>
<https://www.docker.com/>

Contents

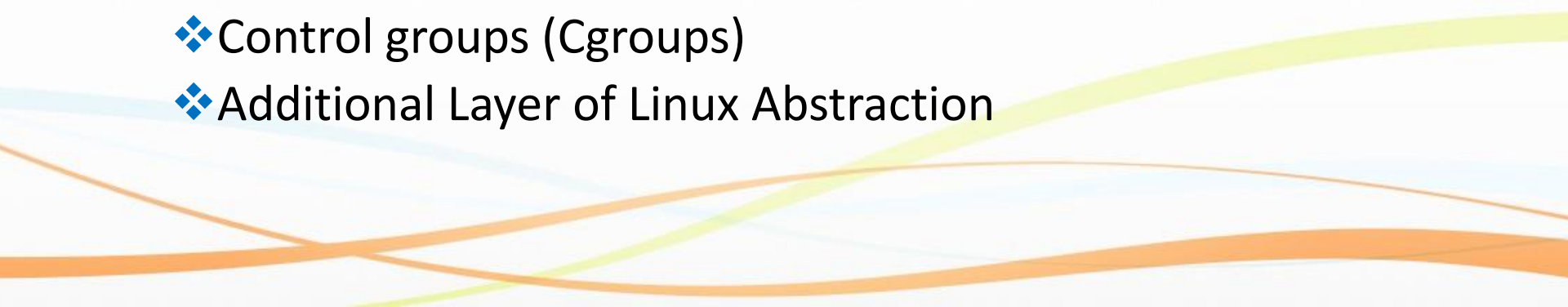
- ▶ Why Docker ?
 - ❖ Standard Deployment Environment
 - ❖ Lightweight Containers
- ▶ What is Docker ?
 - ❖ Standardized Lightweight Container running on Docker Engine
 - ❖ Lightweighted Container to package and run an application
 - ❖ Docker containers are isolated, but share OS and appropriate bins/libraries
- ▶ Docker Platform Architecture
 - ❖ Docker Client, Docker Engine, Docker (Hub) Registry
- ▶ Docker Lifecycle
 - ❖ Pull, Change, Commit, Push
- ▶ Docker Eco System
 - ❖ Many Dockerized Applications, Use Cases, Projects
 - ❖ Technical Support, 3rd Party Tools, Communities
- ▶ Benefits/Drawbacks of Docker
 - ❖ Benefits: Fast Provisioning, Continuous Integration/Deployment, Scale up/down, High Density (can run more workload on the same HW)
 - ❖ Drawbacks: Reduced Isolation and Security

Contents

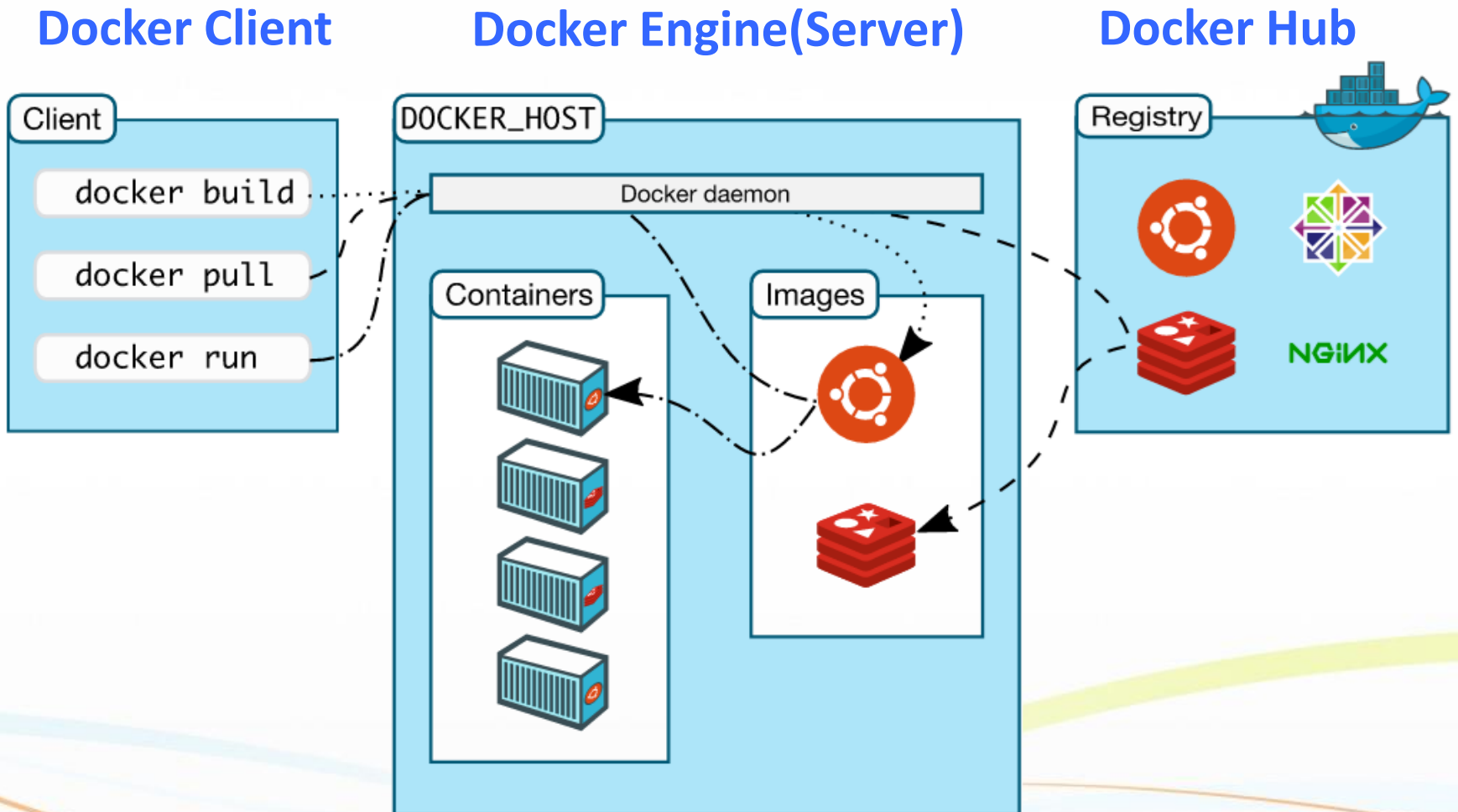
▶ Docker System Architecture

- ❖ Docker Engine
- ❖ Docker Image
- ❖ Docker Container
- ❖ Docker Registry / Hub

▶ Underlying Technologies of Docker

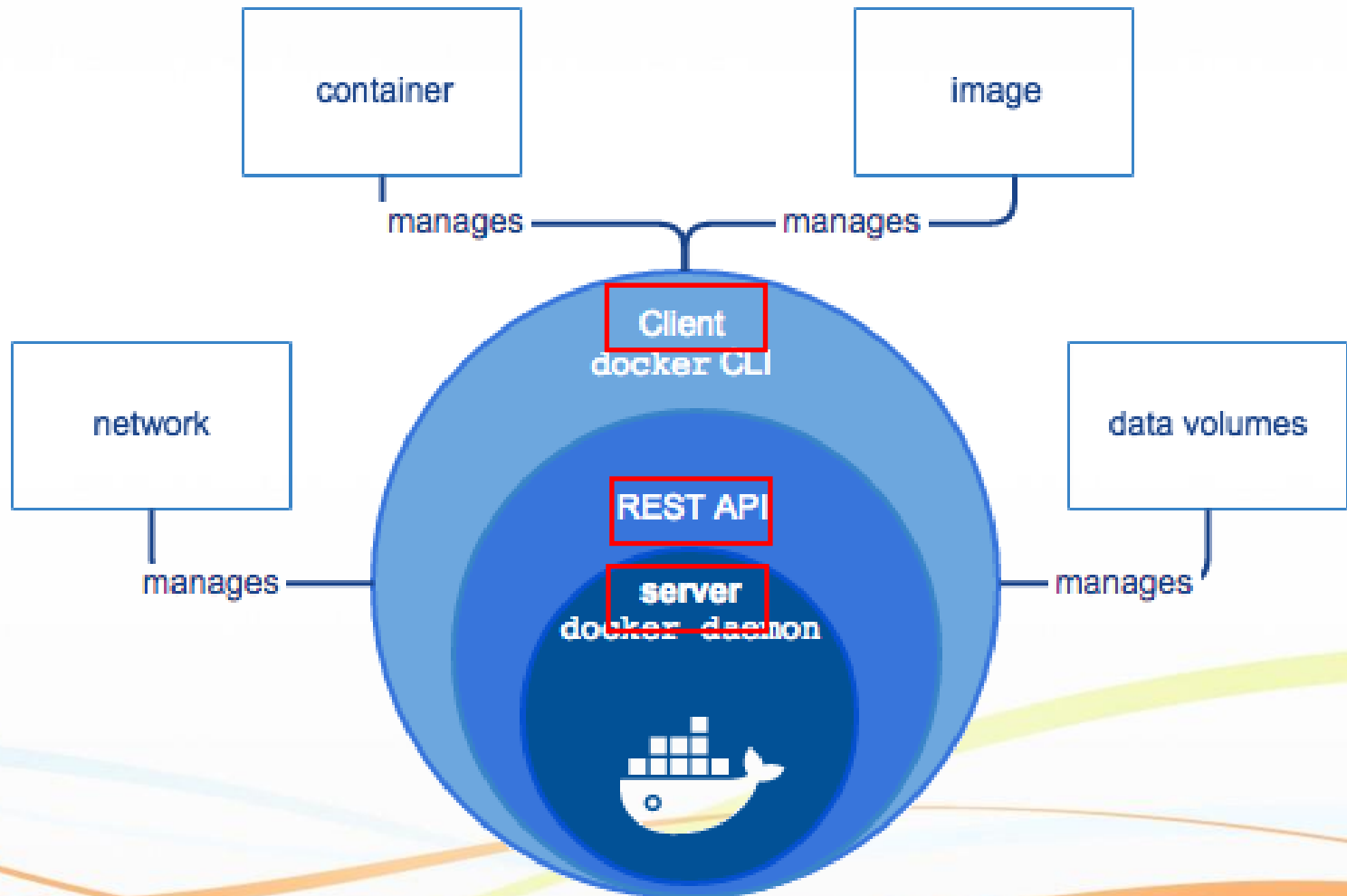
- ❖ Union File System
 - ❖ Namespaces
 - ❖ Control groups (Cgroups)
 - ❖ Additional Layer of Linux Abstraction
- 

Docker Architecture



Docker Engine

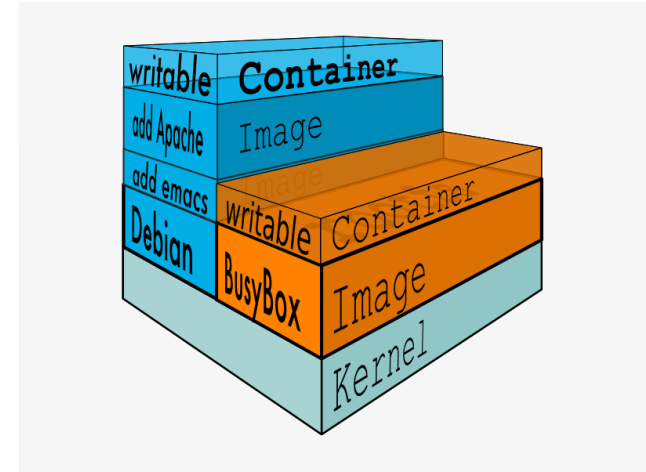
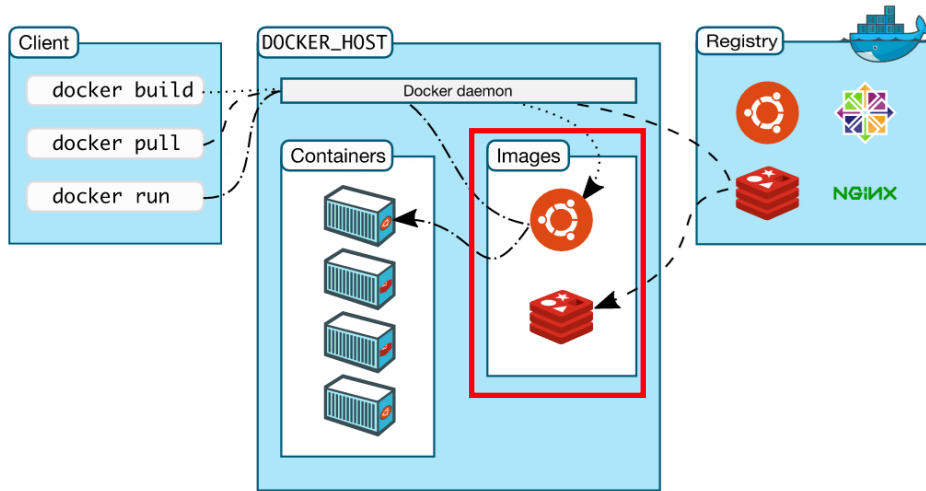
► **Docker Engine** is a **client-server** application



Docker Engine

- ▶ A **server**, a **daemon process**, creates and manages Docker *objects*, such as images, containers, networks, and data volumes.
- ▶ A **REST API** which **specifies interfaces** that programs can use to talk to the daemon and **instruct it what to do**.
- ▶ The **CLI** uses the Docker **REST API to control or interact with the Docker daemon** through scripting or direct CLI commands.
- ▶ Many other Docker applications use the underlying API and CLI.

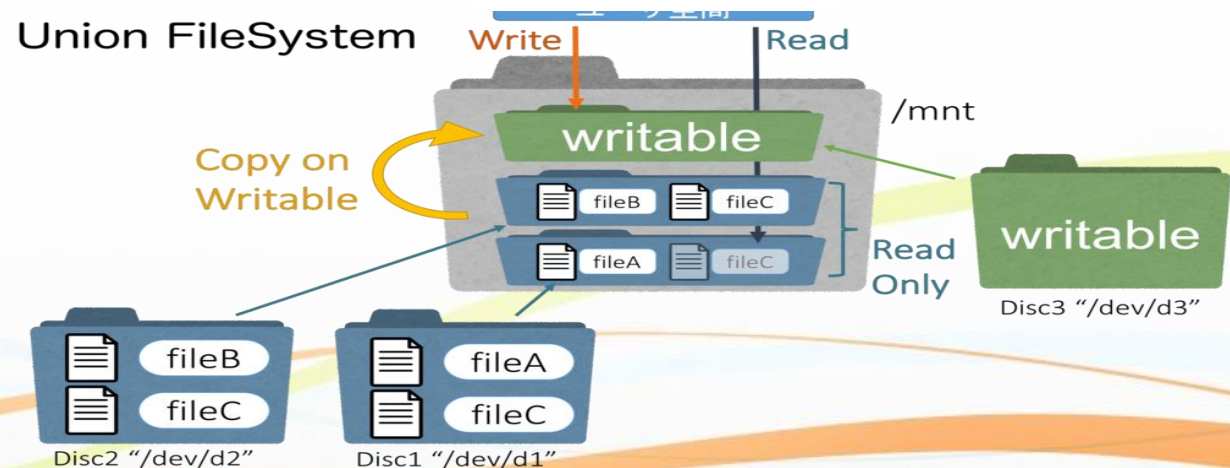
Docker Images



- ▶ A **Docker image** is a **read-only file** used to execute code in a Docker container and act as a set of **instructions for creating a Docker container**.
- ▶ For example, an image might contain an Ubuntu operating system with Apache web server and your web application installed.
- ▶ Each image consists of a series of layers.

Docker Images

- ▶ Docker uses **union file systems** to combine these layers into a single image.
- ❖ Union file systems allow files and directories of separate file systems, known as **branches**, to be transparently overlaid, forming a single coherent file system.
- ❖ When you change a Docker image, such as when you update an application to a new version, a new layer is built and replaces only the layer it updates. The other layers remain intact.



Docker Images

► An image is defined in a **Dockerfile**.

❖ Every image starts from a base image, such as ubuntu, a base Ubuntu image, or a base Fedora image.

```
FROM ubuntu:16.04
```

```
RUN apt-get update -y && \  
    apt-get install -y python-pip python-dev && \  
    pip install --upgrade pip
```

```
COPY ./requirements.txt /app/requirements.txt
```

```
WORKDIR /app
```

```
RUN pip install -r requirements.txt
```

```
COPY . /app
```

```
ENTRYPOINT [ "python" ]
```

```
CMD [ "app.py" ]
```

A Dockerfile

Docker Tutorial <https://www.tutorialspoint.com/docker/index.htm>

→ ↻ 🏠 <https://www.tutorialspoint.com/docker/index.htm>

box (128) - dkmi... [한국대학교 PORTAL](#) [eCampus for KON...](#) [calendar.google.co...](#) [Log In | Microsoft...](#) [한국대학교 - Goog...](#) [Home](#) [넷플릭스](#) [YouTube](#) [생명의 삶](#) [Google Calendar](#) [N](#)

[Home](#) [Coding Ground](#) [Jobs](#) [Whiteboard](#) [Tools](#)

 [Category](#)

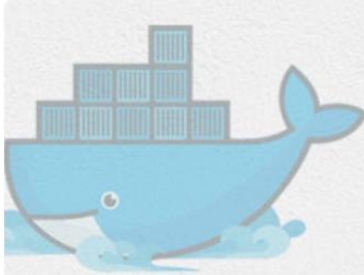
[HTML](#) [CSS](#) [Javascript](#) [SQL](#) [Python](#) [Java](#) [C](#) [C++](#) [PHP](#) [Scala](#) [C#](#) [Node.js](#) [MySQL](#) [MongoDB](#) [PL/SQL](#)



LEARN DOCKER
absolute beginners

Docker Tutorial

- [Docker - Home](#) →
- [Docker - Overview](#)
- [Docker - Installing Docker on Linux](#)
- [Docker - Installation](#)
- [Docker - Hub](#)
- [Docker - Images](#)
- [Docker - Containers](#)
- [Docker - Working With Containers](#)



LEARN DOCKER
simply easy learning

Docker Tutorial

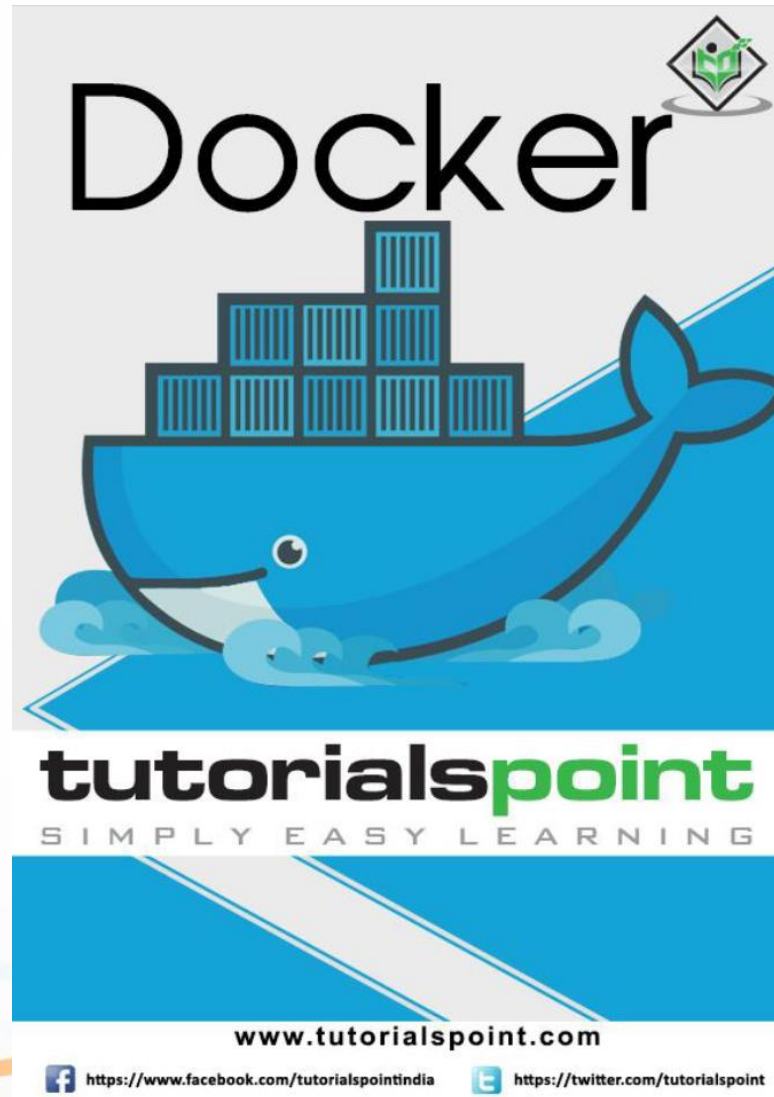
[PDF Version](#) [Quick Guide](#) [Resources](#) [Job Search](#) [Discussion](#)

This tutorial explains the various aspects of the Docker Container service. Starting with the basics of Docker which focuses on the installation and configuration of Docker, it gradually moves on to advanced topics such as Networking and Registries. The last few chapters cover the development aspects of Docker and how you can get up and running on the development environments using Docker Containers.

Audience

This tutorial is meant for those who are interested in learning Docker as a container service. This product has spread like wildfire across the industry and is really making an impact on the development of new generation applications. So anyone who is interested in learning all the aspects of Docker should go through this tutorial.

Docker Tutorial PDF version



Docker Container

- ▶ Docker uses the resource isolation features(cgroups, namespaces) of the Linux kernel to allow independent "containers" to run within a single Linux instance,
 - avoiding the overhead of starting and maintaining virtual machines



- Kernel Feature
- Groups of Processes
- Control Resource Allocation
 - CPU, CPU Sets
 - Memory
 - Disk
 - Block I/O
- The real magic behind containers
- It creates barriers between processes
- Different Namespaces
 - PID Namespace
 - Net Namespace
 - IPC Namespace
 - MNT Namespace
- Linux Kernel Namespace introduced between kernel 2.6.15 – 2.6.26
- Not a File System
- Not a VHD
- Basically a tar file
- Has a Hierarchy
 - Arbitrary Depth
- Fits into Docker Registry

Docker Container

- ▶ A Docker container is a **runnable instance of a Docker image**.
- ▶ You can run, start, stop, move, or delete a container using Docker API or CLI commands.
- ▶ Docker containers wrap up a piece of software in a complete filesystem that contains everything it needs to run:
 - ❖ code, runtime, system tools, system libraries – anything you can install on a server.
- ▶ This guarantees that it will always run the same, regardless of the environment it is running in.

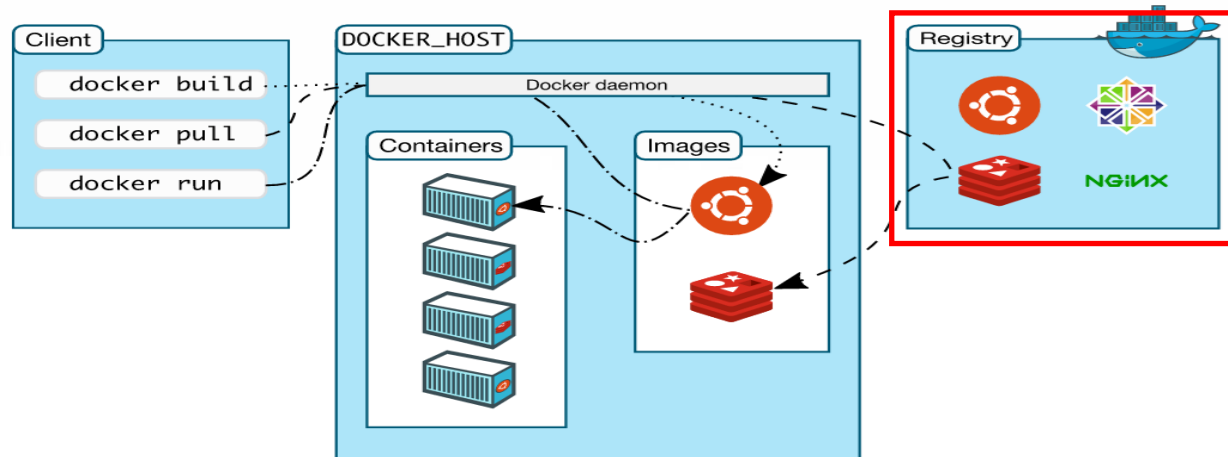
Docker Container

- ▶ The Docker image is read-only. When Docker **runs a container from an image**, it adds a **read-write layer** on top of the image (using a UnionFS as we saw earlier) in which your application runs.
- ▶ An *image* is a filesystem and parameters to use at runtime. It doesn't have state and never changes.
- ▶ A *container* is a **running instance of an image**. When you **run** the command, Docker Engine:
 - ❖ checks to see if you had the application software image
 - ❖ if not, downloads the image from the Docker Hub.
 - ❖ loads the image into the container and “run” it

Docker Container

- ▶ When you run a container, you can **provide configuration metadata** such as networking information or environment variables.
- ▶ Each container is an **isolated and secure application platform**, but can be given access to resources running in a different host or container, as well as persistent storage or databases.
- ▶ A container **uses the host machine's Linux kernel**, and **consists of any extra files you add** when the image is created, along with metadata associated with the container at creation or when the container is started.

Docker Registry / Hub



- ▶ A docker registry is **a library of images**.
- ▶ A registry **can be public or private**, and can be on the same server as the Docker daemon or Docker client, or on a totally separate server.
- ▶ *Docker Hub* is a public registry and stores images.

❖ <https://hub.docker.com/>

Public Docker Hub

<https://hub.docker.com/>

DockerCon 2023: Our annual developer event is back -- online & in person. [Learn more.](#)



[Explore](#) [Repositories](#) [Organizations](#) [Help](#)

[Upgrade](#)  dkmin21

Welcome to Docker

Download the desktop application

[Download for Windows](#)

Also available for [Mac](#) and [Linux](#)



Create a Repository

Push container images to a repository on Docker Hub.



Docker Hub Basics

Watch the guide on how to create and push your first image into a Docker Hub repository.



Language-Specific Guides

Learn how to containerize language-specific applications using Docker.

Access the world's largest library of container images

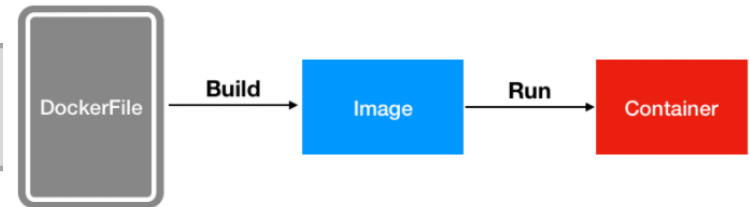
				
 busybox Official 1B+	 ubuntu Official 1B+	 python Official 1B+	 postgres Official 1B+	 httpd Official 1B+
 memcached Official 1B+	 mysql Official 1B+	 traefik Official 1B+	 mariadb Official 1B+	 docker Official 1B+

Docker Commands:

Built image and Run Container

- Run the following command to build the docker image *web* from your directory

```
$ docker build -t web .
```



- Run the Docker Container

```
$ docker run -d -p 5000:5000 web
```

- You can find the container runtime details.

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
9d1c00647a25	web	"python app.py"	53 seconds ago	Up 52 seconds	0.0.0.0:5000->5000/tcp	small_mccarthy

What happens when you run a container?

1. Pulls the ubuntu image :

Docker Engine checks for the presence of the ubuntu image. If the image already exists locally, Docker Engine uses it for the new container. Otherwise, then Docker Engine pulls it from Docker Hub.

2. Creates a new container :

Docker uses the image to create a container.

3. Allocates a filesystem and mounts a read-write layer

: The container is created in the file system and a read-write layer is added to the image.

What happens when you run a container?

4. Allocates a network / bridge interface :

Creates a network interface that allows the Docker container to talk to the local host.

5. Sets up an IP address :

Finds and attaches an available IP address from a pool.

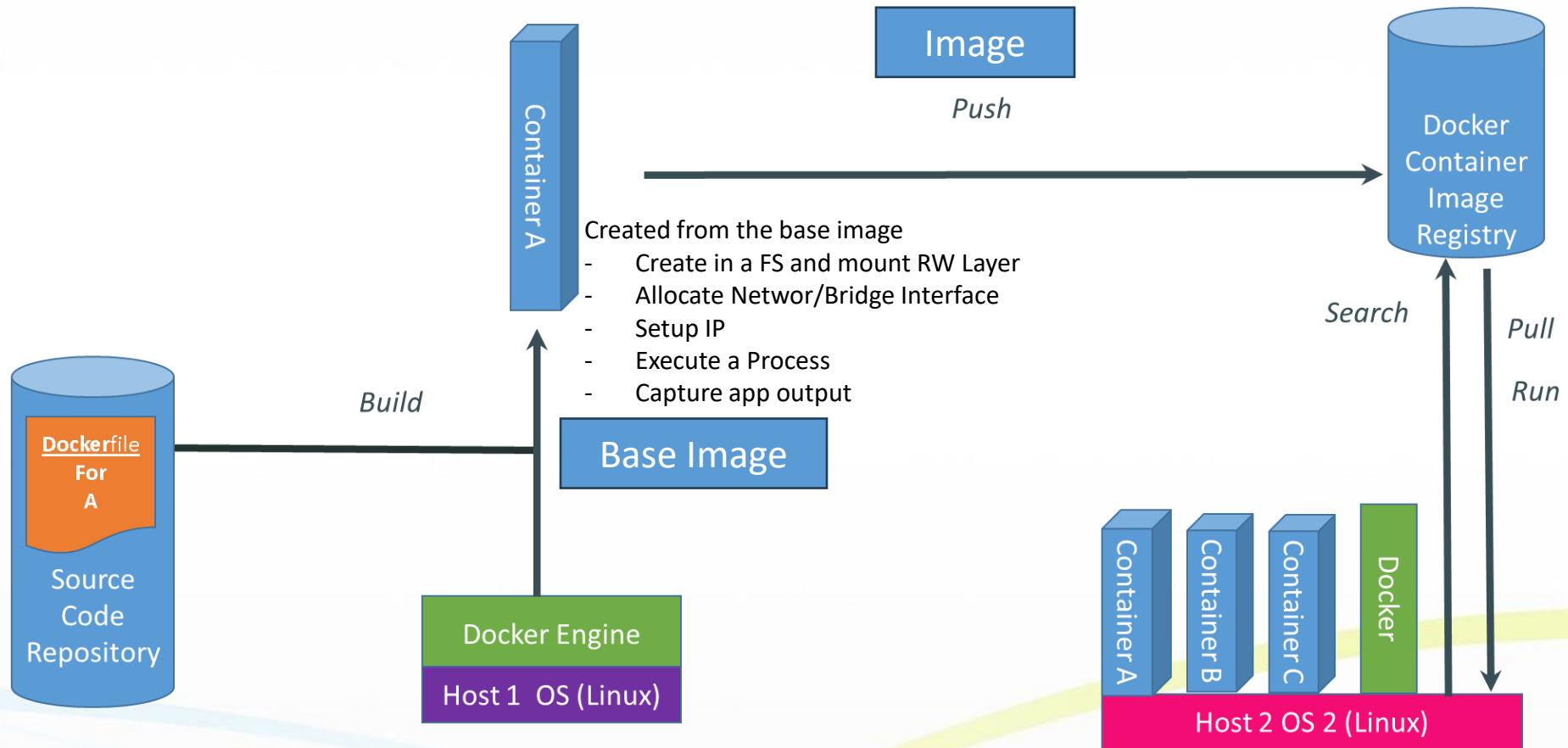
6. Executes a process that you specify :

Executes the /bin/bash executable.

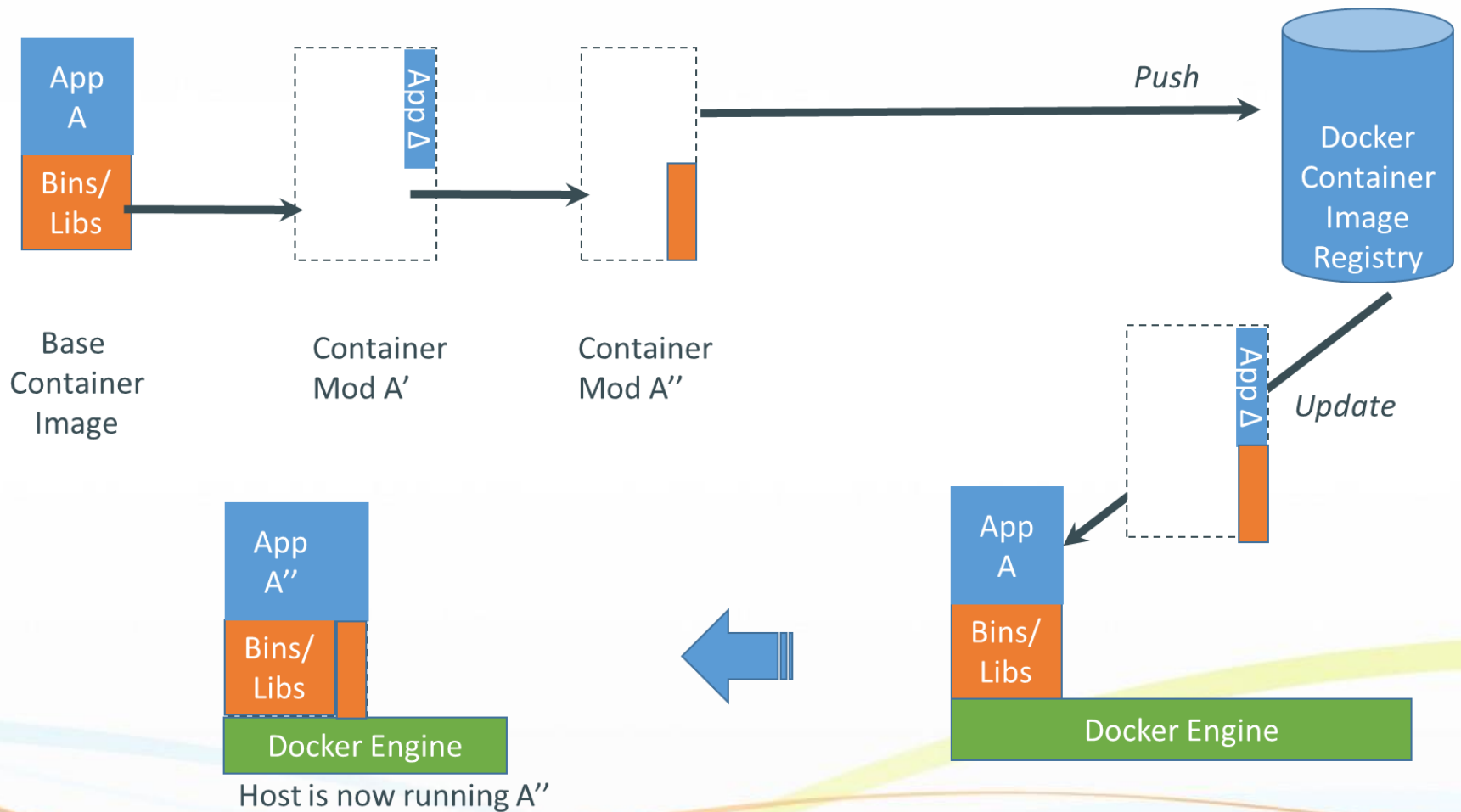
7. Captures and provides application output :

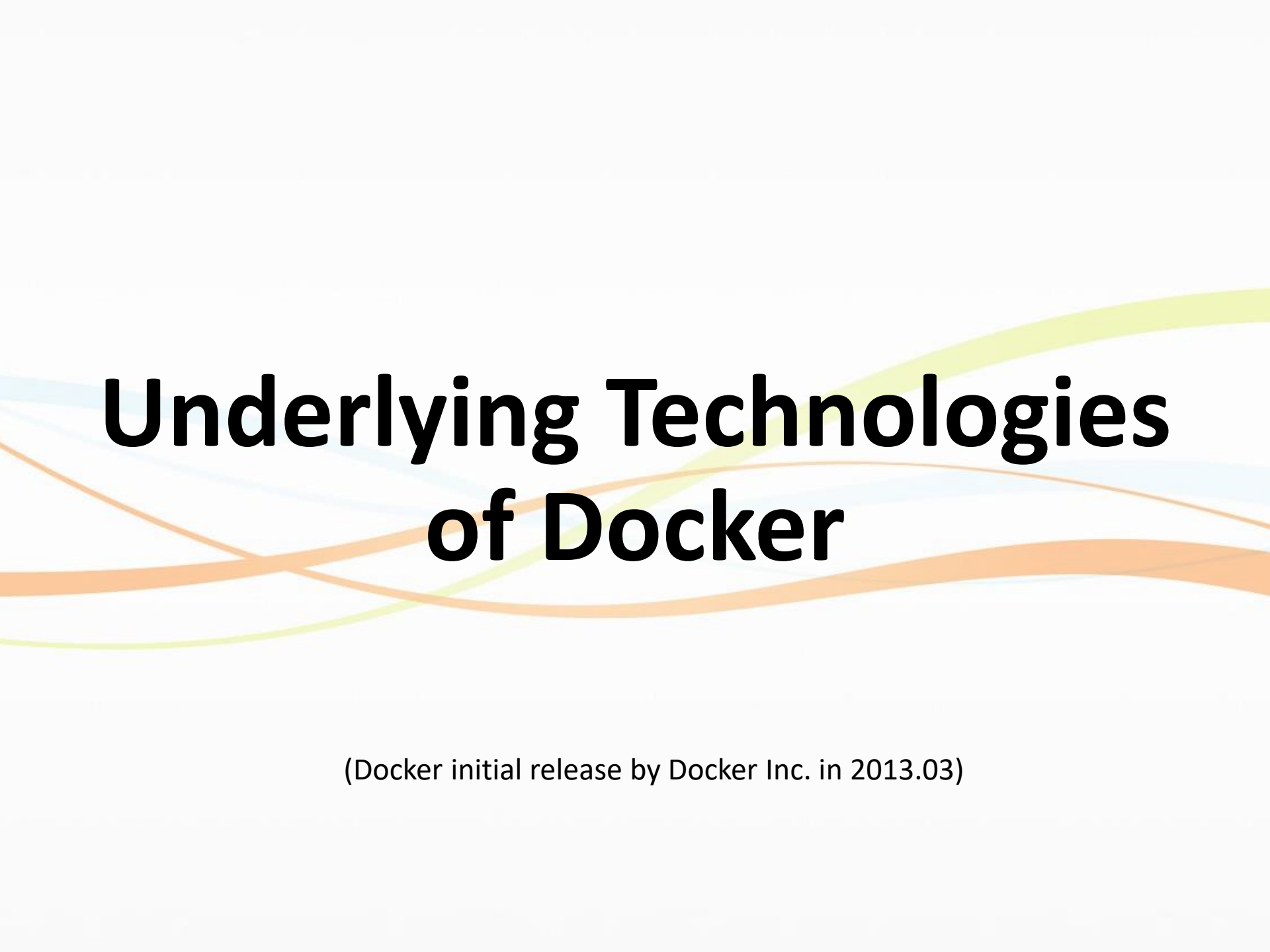
Connects and logs standard input, outputs and errors for you to see how your application is running, because you requested interactive mode.

How they works?



Change and Update

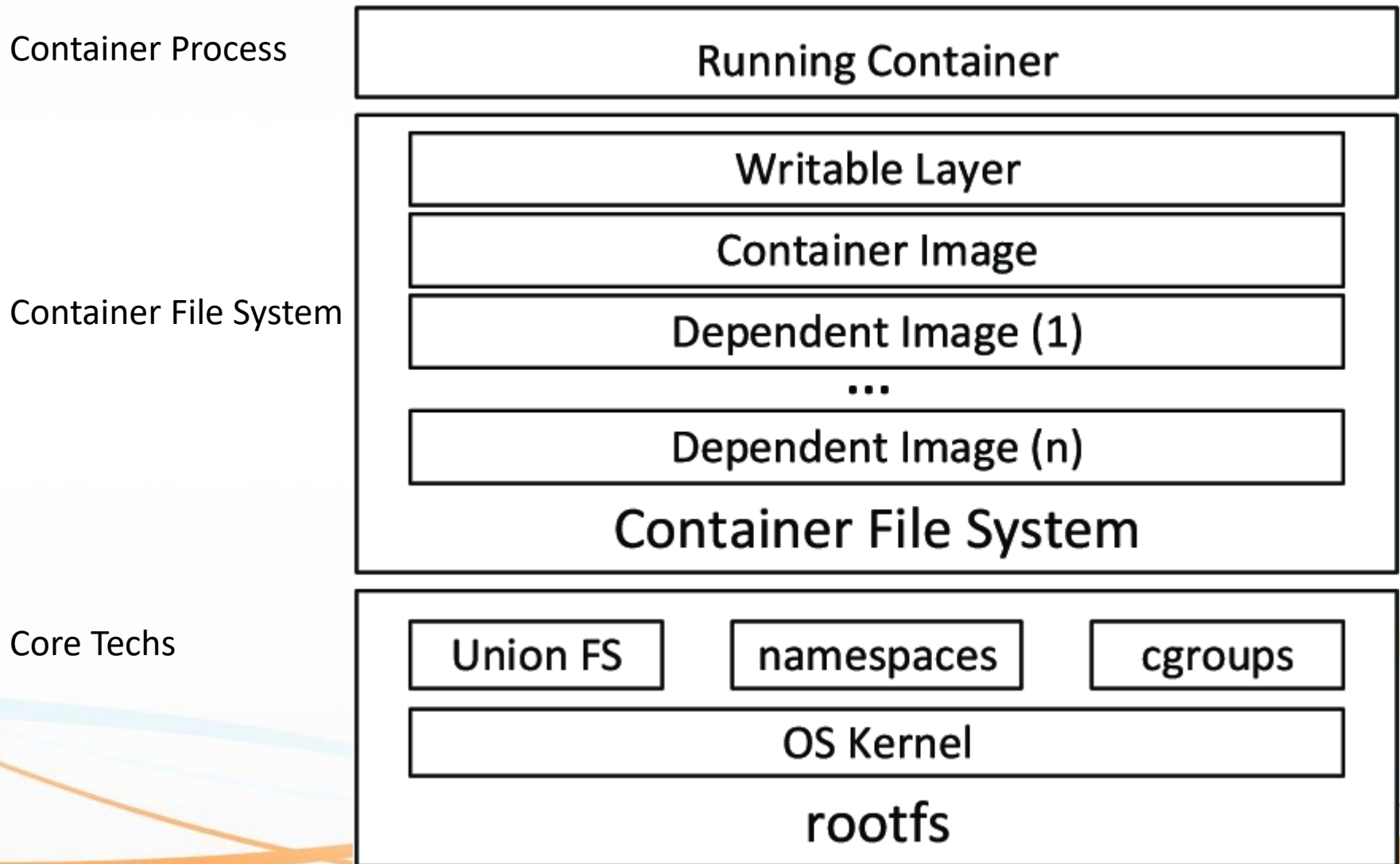




Underlying Technologies of Docker

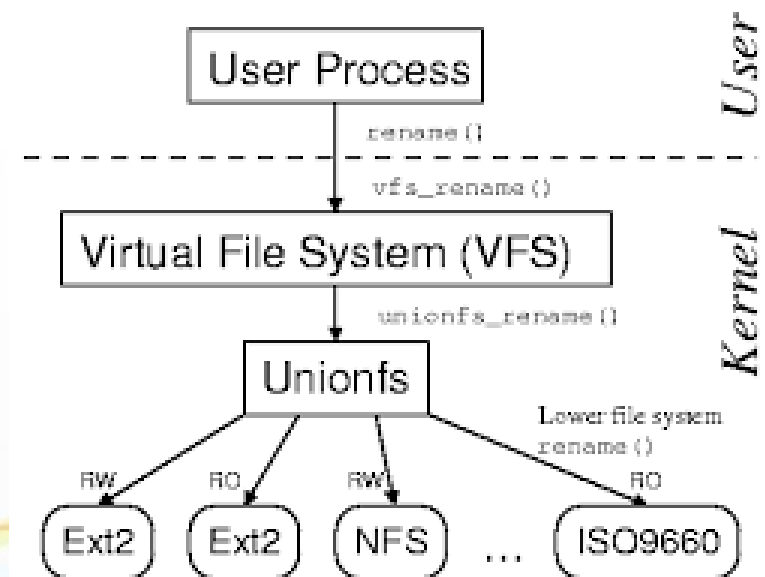
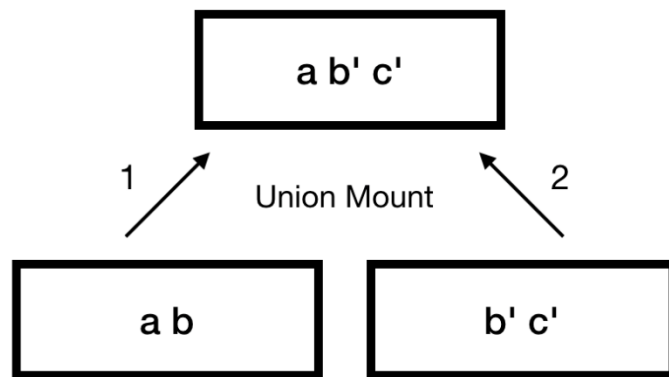
(Docker initial release by Docker Inc. in 2013.03)

A Docker Container



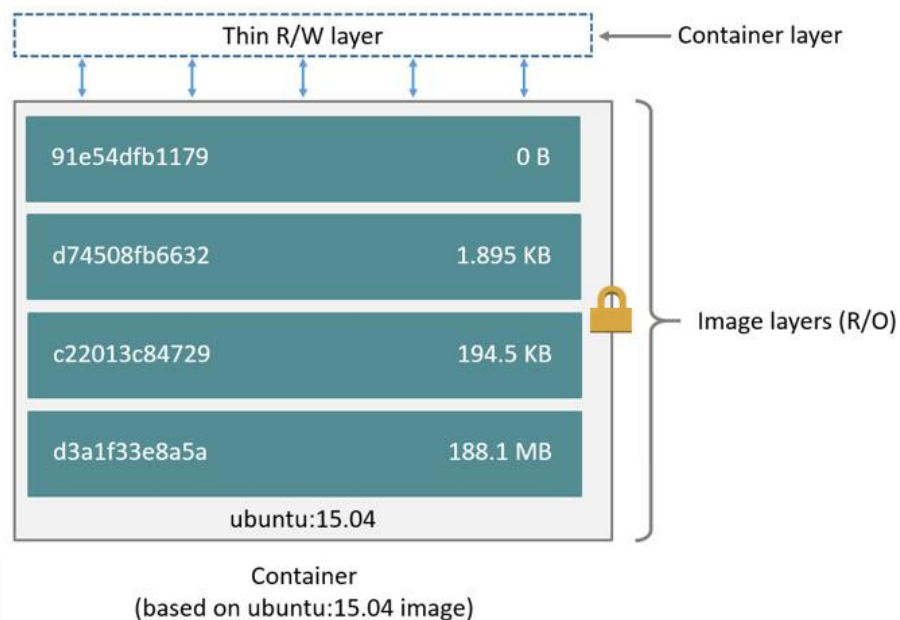
Union File System (paper in 2004.12)

- UnionFS
 - A filesystem that implements a [union mount](#) for other file systems
 - It allows files and directories of separate file systems, known as branches, to be transparently overlaid, forming a single coherent file system

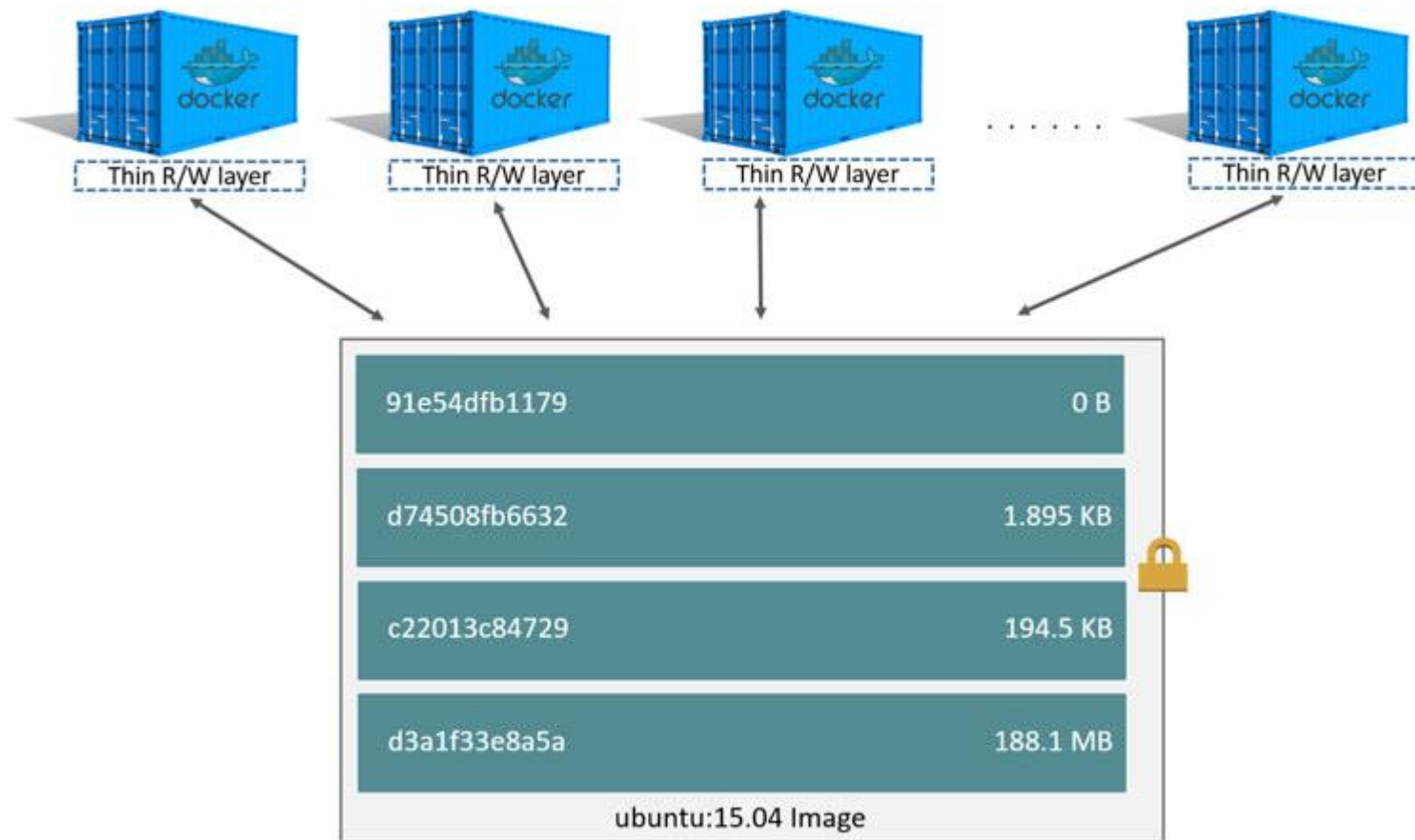


Docker File System (Docker FS)

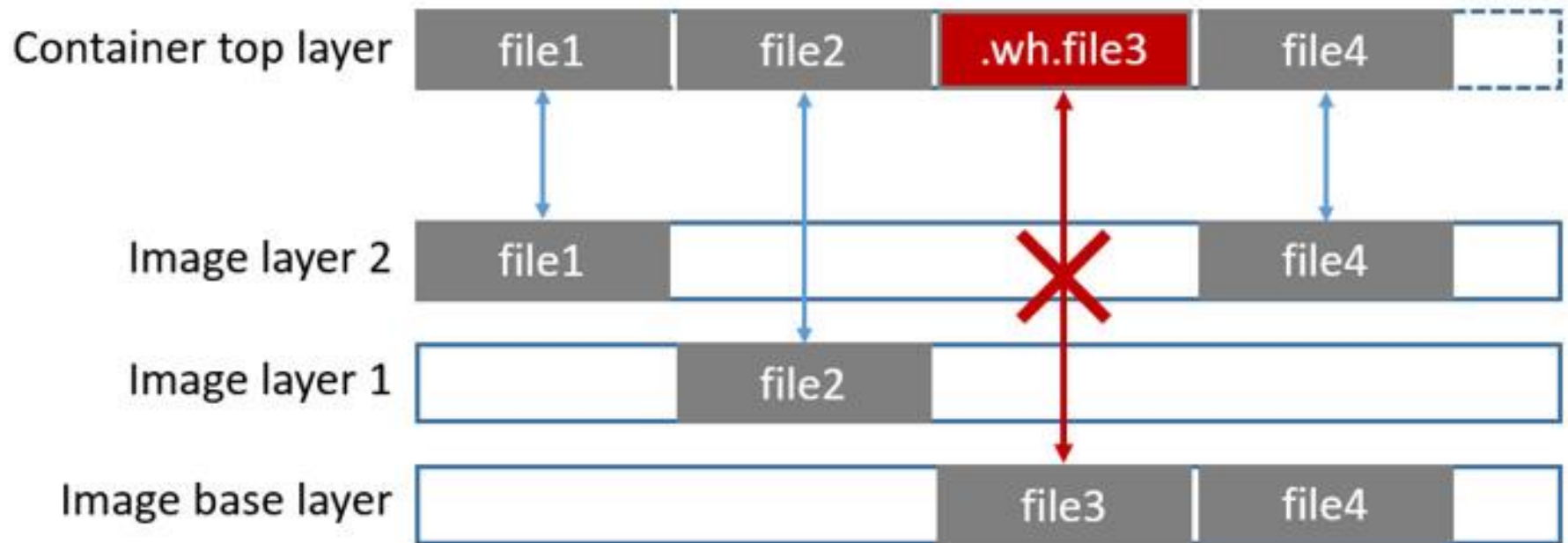
- ▶ Docker Engine uses Union file systems, or UnionFS, which are file systems that **operate by creating layers**, making them very lightweight and fast.
- ▶ Docker Engine can use multiple UnionFS variants, including AUFS, btrfs, vfs, and DeviceMapper.



DockerFS: Sharing Lower FS in RO mode



DockerFS: Copy-on-Write



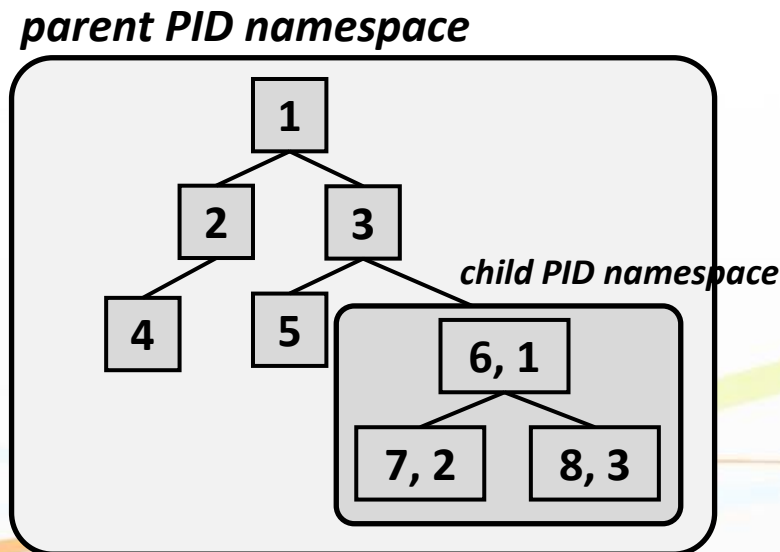
Docker container
(AUFS storage-driver demonstrating whiteout file)

Resource Isolation Technologies

- ▶ Docker uses the resource isolation features of the [Linux kernel](#) to allow independent "containers" to run within a single Linux instance,
 - avoiding the overhead of starting and maintaining [virtual machines](#)
- ❖ [Namespaces](#) (start from linux 2.4.19, 2002)
 - isolates an application's view of the operating environment, including process trees, network, user IDs and mounted file systems,
- ❖ [cgroups](#) (start from linux 2.6.2, 2007)
 - provide resource limiting, including the CPU, memory, block I/O, and network.

Namespace

- ▶ Historically, the Linux kernel has maintained a **single process tree**. The tree contains a reference to every process currently running in a parent-child hierarchy.
- ▶ With the introduction of Linux namespaces, it became possible to have **multiple “nested” process trees**.



Namespace

▶ Three system calls are used for namespaces:

- ❖ **clone()** - creates a new process and a new namespace; the process is attached to the new namespace.

- Process creation and process termination methods, `fork()` and `exit()` methods, were patched to handle the new namespace `CLONE_NEW*` flags.

- ❖ **unshare()** - does not create a new process; creates a new namespace and attaches the current process to it.

- The `unshare()` system call was added in 2005:

- ❖ **setns(int fd, int nstype)** - a new system call was added, for joining an existing namespace. Available only from kernel 3.0

Docker Namespace

- ▶ Docker uses **namespaces** to provide the isolated workspace called “container”.
- ▶ When you run a container, Docker creates a set of 6 namespaces for that container.
 - ❖ mnt (mount points, filesystems)
 - ❖ pid (processes)
 - ❖ net (network stack)
 - ❖ ipc (System V IPC)
 - ❖ uts (hostname)
 - ❖ user (UIDs)
- ▶ These namespaces **provide a layer of isolation**. Each aspect of a container runs in a separate namespace and its access is limited to that namespace.

Control groups (Cgroups)

CPU Scheduling
Virtual Memory Management
Control usage of IO,Net

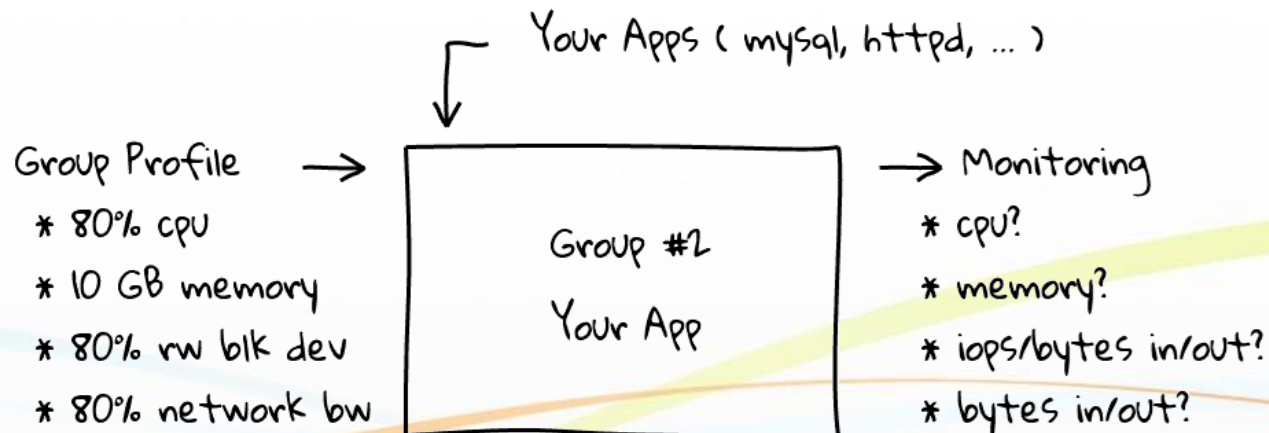
- ▶ **cgroups** (control groups) subsystem is a **Resource Management solution** providing a **generic process grouping framework**.
- ▶ This work was started by developers from Google (primarily Paul Menage and Rohit Seth) in **2006** under the name "**process containers**"; in **2007**, renamed to "**Control Groups**"
 - ❖ Maintainers: Li Zefan (Huawei) and Tejun Heo.
 - ❖ The **memory controller** (memcg), which is probably it is the most complex, is maintained separately (4 maintainers)

Control groups (Cgroups)

- ▶ Docker Engine on Linux also relies on another technology called control groups (cgroups).
 - ▶ A cgroup **limits an application to a specific set of resources**.
 - ▶ Control groups allow Docker Engine to share available hardware resources to containers and optionally enforce limits and constraints.
 - ▶ For example, you can limit the memory available to a specific container.
- 

Control groups (Cgroups)

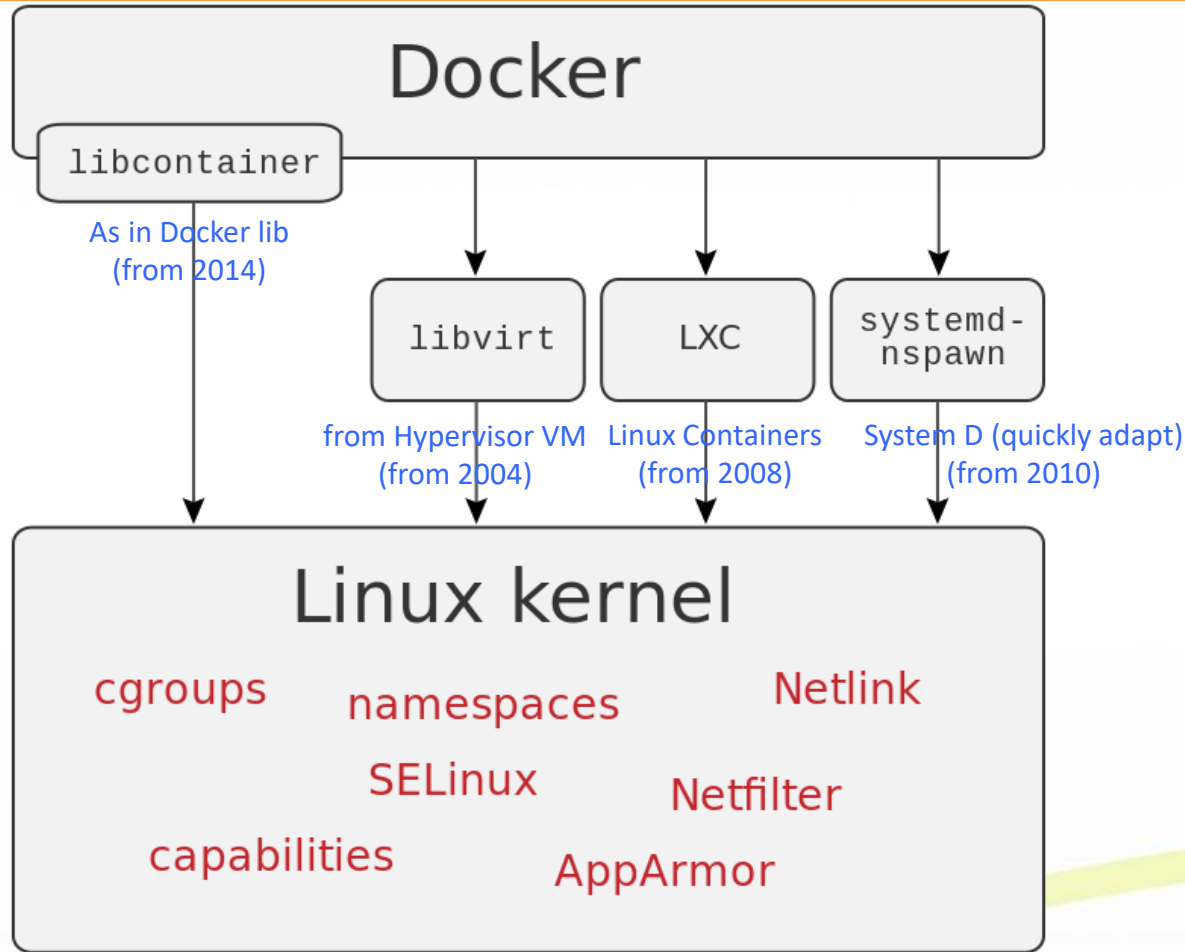
- ▶ Lets say for example that you have a resources intensive application on a server.
- ▶ Linux is great at sharing resources between all of the processes on a system, but in some cases, **you want to allocate, or guarantee, a greater amount to a specific application, or a set of applications**, this is where control groups are useful.



Namespaces and Cgroups

- ▶ **Namespaces** provide a **per process resource isolation solution**.
- ▶ **Cgroups**, on the other hand, provides **resource management solution** (handling groups).

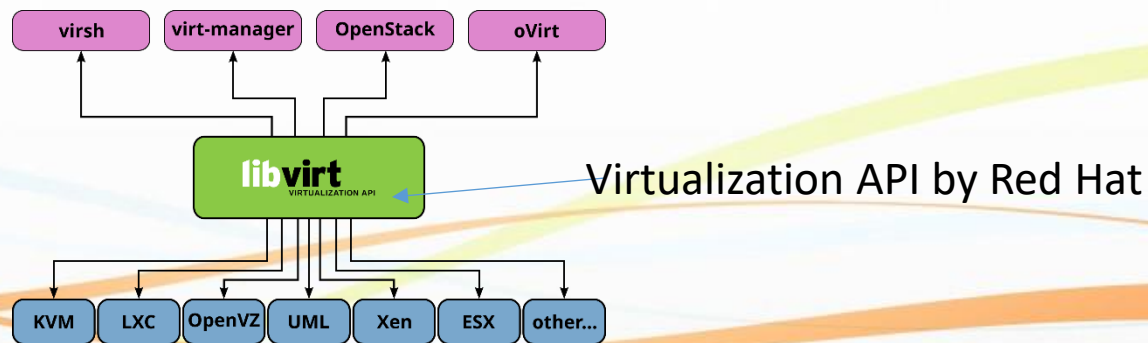
Additional Layer of Abstraction



Docker can use different interfaces to access virtualization features of the Linux kernel

Additional Layer of Abstraction

- ▶ Docker provides an additional layer of abstraction and automation of operating-system-level virtualization on Windows and Linux.
 - ❖ Docker can use different interfaces to access virtualization features of the Linux kernel.¹
 - ❖ Since version 0.9, Docker includes the libcontainer library as its own way to directly use virtualization facilities provided by the Linux kernel,
 - ❖ in addition to using abstracted virtualization interfaces via libvirt, LXC (Linux Containers) and systemd-nspawn



Summary

▶ Docker System Architecture

- ❖ Docker Host: Docker Server(Daemon), REST API, CLI

- ❖ Docker Image

- a read-only file composed of a series of layers based on Union File Systems
- defined in a Dockefile

- ❖ Docker Container

- A runnable instance of a docker image
 - Based on Cgroups, Namespaces, Images
- 

Summary

▶ Underlying Techs in Docker

❖ Union File System

- a filesystem that implements a [union mount](#) for other file systems
- a top writable layer + lower RO layers
- copy-on-write when modifying

❖ Resource Isolation Techs

➤ Namespaces

- provide the isolated workspace called “container”
- A container runs in and limit to a separate namespace

➤ Cgroups

- A resource management solution providing a generic process grouping framework
- allows docker engine to share available hardware resources to containers and optionally enforce limits and constraints.

❖ Additional Layer of Abstraction

- to access virtual features of the Linux Kernel

Thank You!!



Any Questions ??